

제05장.

- 오라클 접속에서 새로운 유저 생성

```
create user user05 identified by pass;  
grant connect, resource, dba to user05;
```

- USER04 접속 SQL 워크시트에서 출력 활성화

```
SET SERVEROUTPUT ON;
```

- 학사관리 통계

1) 중첩테이블을 사용하여 교수 테이블에서 학과별 직급별 인원수를 구하여 테이블에 저장한다.

```
CREATE OR REPLACE PROCEDURE EX01  
AUTHID CURRENT_USER --프로시저를 사용하는 현재 사용자에게 권한 부여  
AS  
    --중첩테이블 타입선언  
    TYPE type_dept IS TABLE OF professors.dept%TYPE;  
    TYPE type_title IS TABLE OF professors.title%TYPE;  
  
    --중첩테이블 변수선언  
    array_dept type_dept;  
    array_title type_title := type_title('정교수', '부교수', '조교수');  
  
    --변수 선언  
    v_sql VARCHAR2(1000);  
    v_count INT;  
BEGIN  
    BEGIN  
        EXECUTE IMMEDIATE 'DROP TABLE tbl_temp';  
    EXCEPTION  
        WHEN OTHERS THEN  
            IF SQLCODE != -942 THEN RAISE; END IF; --테이블 없음 에러가 아니면 에러를 발생시킴  
    END;  
    EXECUTE IMMEDIATE 'CREATE TABLE tbl_temp(dept VARCHAR2(100), title VARCHAR2(100), cnt NUMBER)';  
  
    -- 결과를 BULK COLLECT를 사용하여 한 번에 중첩테이블 변수에 담아 성능 최적화  
    SELECT DISTINCT(dept) BULK COLLECT INTO array_dept FROM professors;  
    FOR i IN 1..array_dept.COUNT LOOP  
        --DBMS_OUTPUT.PUT_LINE('학과명:' || array_dept(i));  
        FOR j IN 1..array_title.COUNT LOOP  
            --DBMS_OUTPUT.PUT_LINE('직급명:' || array_title(j));  
            SELECT COUNT(*) INTO v_count FROM professors WHERE dept=array_dept(i) AND title=array_title(j);  
            DBMS_OUTPUT.PUT_LINE('학과명:' || array_dept(i) || ',직급:' || array_title(j) || ',인원수:' || v_count);  
            v_sql := 'INSERT INTO tbl_temp(dept, title, cnt) VALUES(:1, :2, :3)';  
            EXECUTE IMMEDIATE v_sql USING array_dept(i), array_title(j), v_count;  
        END LOOP;  
    END LOOP;  
END EX01;
```

```
exec ex01();
```

2) 중첩테이블을 사용하여 학생 테이블에서 학과별 학년별 인원수를 구하여 테이블에 저장한다.

```
CREATE OR REPLACE PROCEDURE EX02
AUTHID CURRENT_USER
AS
    TYPE type_dept IS TABLE OF students.dept%TYPE;
    TYPE type_year IS TABLE OF students.year%TYPE;
    array_dept type_dept;
    array_year type_year := type_year(1, 2, 3, 4);
    v_sql VARCHAR2(1000);
    v_count INT;
BEGIN
    BEGIN
        EXECUTE IMMEDIATE 'DROP TABLE tbl_temp';
    EXCEPTION
        WHEN OTHERS THEN
            IF SQLCODE != -942 THEN RAISE; END IF;
    END;
    EXECUTE IMMEDIATE 'CREATE TABLE tbl_temp(dept VARCHAR2(100), year INT, cnt NUMBER)';

    SELECT DISTINCT(dept) BULK COLLECT INTO array_dept FROM students;
    FOR i IN 1..array_dept.COUNT LOOP
        FOR j IN 1..array_year.COUNT LOOP
            SELECT COUNT(*) INTO v_count FROM students WHERE dept=array_dept(i) AND year=array_year(j);
            DBMS_OUTPUT.PUT_LINE('학과명:' || array_dept(i) || ',학년:' || array_year(j) || '인원:' || v_count);
            v_sql := 'INSERT INTO tbl_temp(dept, year, cnt) VALUES(:1, :2, :3)';
            EXECUTE IMMEDIATE v_sql USING array_dept(i), array_year(j), v_count;
        END LOOP;
    END LOOP;
END EX02;
```

```
exec ex02();
```

3) 시작일과 종료일을 입력 받아 일별 수강 신청 인원수를 출력한다.

```
CREATE OR REPLACE PROCEDURE EX03(
    i_sdate IN enrollments.edate%TYPE,
    i_edate IN enrollments.edate%TYPE)
AS
    v_sdate DATE := TO_DATE(i_sdate, 'YY/MM/DD');
    v_edate DATE := TO_DATE(i_edate, 'YY/MM/DD');
    v_count INT;
BEGIN
    WHILE v_sdate <= v_edate LOOP
        --DBMS_OUTPUT.PUT_LINE(TO_CHAR(v_sdate, 'YY/MM/DD'));
        SELECT COUNT(*) INTO v_count FROM enrollments WHERE edate=TO_CHAR(v_sdate, 'YY/MM/DD');
        DBMS_OUTPUT.PUT_LINE('수강일:' || TO_CHAR(v_sdate, 'YY/MM/DD') || ', 신청인원:' || v_count);
        v_sdate := v_sdate + 1; --하루씩 증가
    END LOOP;
END EX03;
```

```
exec ex03('98/03/01', '98/03/06');
```

4) 학생별 평균점수와 수강신청한 과목수를 출력한다.

```
CREATE OR REPLACE PROCEDURE EX04
AS
BEGIN
    FOR s IN (SELECT * FROM students ORDER BY scode) LOOP
        --DBMS_OUTPUT.PUT_LINE(rec.scode || ',' || rec.sname);
        FOR e IN (SELECT NVL(AVG(grade), 0) avg_grade, COUNT(*) cnt FROM enrollments WHERE scode=s.scode) LOOP
            DBMS_OUTPUT.PUT_LINE('학번:' || s.scode || ', 이름:' || s.sname ||
                ', 평균:' || TO_CHAR(e.avg_grade, '990.99') || ', 과목수:' || e.cnt);
        END LOOP;
    END LOOP;
END EX04;
```

```
exec ex04();
```

5) 강좌별 평균점수와 수강인원 수를 출력한다.

```
CREATE OR REPLACE PROCEDURE EX05 AS
BEGIN
    FOR c IN (SELECT * FROM courses ORDER BY lcode) LOOP
        --DBMS_OUTPUT.PUT_LINE(c.lcode || ',' || c.lname);
        FOR e IN (SELECT NVL(AVG(grade), 0) avg_grade, COUNT(*) cnt FROM enrollments WHERE lcode=c.lcode) LOOP
            DBMS_OUTPUT.PUT_LINE('강좌번호:' || c.lcode || ', 강좌이름:' || c.lname ||
                ', 평균:' || TO_CHAR(e.avg_grade, '990.99') || ', 수강인원:' || e.cnt);
        END LOOP;
    END LOOP;
END EX05;
```

```
exec ex05();
```

6) 교수별 담당과목 수와 지도 학생 수를 출력한다.

```
create or replace PROCEDURE EX06
AS
    v_lcount INT;
    v_scount INT;
BEGIN
    FOR p IN (SELECT * FROM professors ORDER BY pcode) LOOP
        --DBMS_OUTPUT.PUT_LINE('교수번호:' || p.pcode || ', 교수이름:' || p.pname);
        FOR c IN (SELECT COUNT(*) AS cnt FROM courses WHERE instructor=p.pcode) LOOP
            --DBMS_OUTPUT.PUT_LINE('담당과목수:' || c.cnt);
            v_lcount := c.cnt;
        END LOOP;
        FOR s IN (SELECT COUNT(*) AS cnt FROM students WHERE advisor=p.pcode) LOOP
            --DBMS_OUTPUT.PUT_LINE('지도학생수:' || s.cnt);
            v_scount := s.cnt;
        END LOOP;
        DBMS_OUTPUT.PUT_LINE('교수번호:' || p.pcode || ', 교수이름:' || p.pname ||
            ', 담당과목수:' || v_lcount || ', 지도학생수:' || v_scount);
    END LOOP;
END EX06;
```

```
exec ex05();
```

7) 교수별 해당 교수가 강의하는 강좌수와 이 강좌들을 수강하는 수강 총 인원수를 출력한다.

```
CREATE OR REPLACE PROCEDURE EX07 AS
    TYPE type_professors IS TABLE OF professors%ROWTYPE;
    array_professors type_professors;
    v_total INT;
    v_count INT;
BEGIN
    SELECT * BULK COLLECT INTO array_professors FROM professors;
    FOR i IN 1..array_professors.COUNT LOOP
        v_total :=0;
        v_count :=0;
        --DBMS_OUTPUT.PUT_LINE('교수번호:' || array_professors(i).pcode || ', 교수이름:' || array_professors(i).pname);
        FOR c IN (SELECT * FROM courses WHERE instructor=array_professors(i).pcode) LOOP
            --DBMS_OUTPUT.PUT_LINE('강좌번호:' || c.lcode || ', 강좌이름:' || c.lname);
            v_count := v_count + 1;
            FOR e IN (SELECT COUNT(*) AS cnt FROM enrollments WHERE lcode=c.lcode) LOOP
                --DBMS_OUTPUT.PUT_LINE('수강인원:' || e.cnt);
                v_total := v_total + e.cnt;
            END LOOP;
        END LOOP;
        DBMS_OUTPUT.PUT_LINE('교수번호:' || array_professors(i).pcode || ', 강좌수:' || v_count || ', 수강인원:' || v_total);
        DBMS_OUTPUT.PUT_LINE('-----');
    END LOOP;
END EX07;
```

```
exec ex07();
```

8) 강의실별 해당 강의실에서 강의하는 강좌수와 이 강좌들을 수강하는 수강 총 인원수를 출력한다.

```
CREATE OR REPLACE PROCEDURE EX08 AS
    TYPE type_room IS TABLE OF courses.room%TYPE;
    array_room type_room;
    v_total INT;
    v_count INT;
BEGIN
    SELECT DISTINCT(room) BULK COLLECT INTO array_room FROM courses WHERE room IS NOT NULL;
    FOR i IN 1..array_room.COUNT LOOP
        v_total := 0;
        v_count := 0;
        --DBMS_OUTPUT.PUT_LINE('강의실:' || array_room(i));
        FOR c IN (SELECT * FROM courses WHERE room=array_room(i)) LOOP
            --DBMS_OUTPUT.PUT_LINE('강좌번호:' || c.lcode);
            v_count := v_count + 1;
            FOR e IN (SELECT COUNT(*) as cnt FROM enrollments WHERE lcode=c.lcode) LOOP
                v_total := v_total + e.cnt;
                --DBMS_OUTPUT.PUT_LINE('수강인원:' || e.cnt);
            END LOOP;
        END LOOP;
        DBMS_OUTPUT.PUT_LINE('강의실:' || array_room(i) || ', 강좌수:' || v_count || ', 수강인원:' || v_total);
        DBMS_OUTPUT.PUT_LINE('-----');
    END LOOP;
END EX08;
```

```
exec ex08();
```

- Trigger(트리거)

트리거(Trigger)는 말 그대로 '방아쇠'라는 뜻으로, 데이터베이스에 특정 이벤트(INSERT, UPDATE, DELETE)가 발생했을 때 자동으로 실행되도록 설정한 PL/SQL 블록이다. 데이터 무결성을 지키고 복잡한 비즈니스 규칙을 자동화하기 위해 사용한다.

- 로그 기록(History 관리)

1) 교수 테이블의 교수정보가 수정된 후 수정하기 전 교수 정보들을 교수 History 테이블에 저장한다.

```
--professors 테이블 구조만 professors_history로 복사
create table professors_history as
    select * from professors where pcode is null;
--professors_history 테이블에 change_date 컬럼 추가
alter table professors_history add change_date timestamp;
--professors_history 테이블 삭제
drop table professors_history;
```

```
CREATE OR REPLACE TRIGGER TRG01
AFTER UPDATE ON PROFESSORS --변경 후 실행
FOR EACH ROW --영향을 받는 행마다 트리거 로직 실행
BEGIN
    INSERT INTO professors_history(pcode, pname, dept, hiredate, title, salary, change_date)
    VALUES(:OLD.pcode, :OLD.pname, :OLD.dept, :OLD.hiredate, :OLD.title, :NEW.salary, SYSDATE);
END;
```

```
update professors set salary=salary + 100000 where pcode='228';
```

2) 수강신청 테이블의 데이터가 변경되면 변경된 행들의 데이터들을 수강신청 History 테이블에 저장한다.

```
create table enrollments_history
as select * from enrollments where lcode is null;
alter table enrollments_history add change_date timestamp add state varchar(20);
drop table enrollments_history;
```

```
CREATE OR REPLACE TRIGGER TRG02
AFTER DELETE OR INSERT OR UPDATE ON ENROLLMENTS
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        INSERT INTO enrollments_history(lcode, scode, edate, grade, change_date, state)
        VALUES(:NEW.lcode, :NEW.scode, :NEW.edate, :NEW.grade, SYSDATE, 'INSERT');
    ELSIF DELETING THEN
        INSERT INTO enrollments_history(lcode, scode, edate, grade, change_date, state)
        VALUES(:OLD.lcode, :OLD.scode, :OLD.edate, :OLD.grade, SYSDATE, 'DELETE');
    ELSIF UPDATING THEN
        INSERT INTO enrollments_history(lcode, scode, edate, grade, change_date, state)
        VALUES(:OLD.lcode, :OLD.scode, :OLD.edate, :OLD.grade, SYSDATE, 'UPDATE_OLD');
        INSERT INTO enrollments_history(lcode, scode, edate, grade, change_date, state)
        VALUES(:NEW.lcode, :NEW.scode, :NEW.edate, :NEW.grade, SYSDATE, 'UPDATE_NEW');
    END IF;
END;
```

```
insert into enrollments(lcode, scode, edate) values('A109', '92414033', sysdate);
update enrollments set grade=90 where lcode='A109' and scode='92414033';
delete from enrollments where lcode='A109' and scode='92414033';
```

• 값 자동 증가 (Auto Increment)

1) 수강신청 테이블에 새로운 칼럼(rn)을 추가하고 이 칼럼에 행 번호(rownum)으로 수정한다.

```
alter table enrollments add rn int;
update enrollments set rn=rownum;
select * from enrollments order by rn desc;
```

2) 새로운 시퀀스(seq_enroll_rn)를 26부터 1씩 증가하도록 생성한다.

```
create sequence seq_enrollments start with 26 increment by 1;
drop sequence seq_enrollments;
```

3) 수강신청 테이블에 데이터를 입력하기 전에 행 번호(rn) 칼럼에 새로운 시퀀스 증가 값을 저장한다.

```
CREATE OR REPLACE TRIGGER TRG03
BEFORE INSERT ON ENROLLMENTS
FOR EACH ROW
BEGIN
    SELECT seq_enrollments.NEXTVAL INTO :NEW.rn FROM DUAL;
END;
```

```
insert into enrollments(lcode, scode, edate) values('A109', '92414033', sysdate);
```

• 데이터 보안과 무결성

1) 목요일에는 수강신청 테이블(enrollments) 데이터를 삭제하지 못하도록 제한 한다

```
CREATE OR REPLACE TRIGGER TRG04
BEFORE DELETE ON ENROLLMENTS
BEGIN
    --DY(월/MON), DAY(월요일/MONDAY), D(요일 숫자:1=일요일~7=토요일)
    IF TO_CHAR(SYSDATE, 'DY', 'NLS_DATE_LANGUAGE=KOREAN') IN ('목') THEN
        --트랜잭션을 중단시키고 롤백을 유도하며 에러번호는 -20000부터 -20999사이의 번호 지정
        RAISE_APPLICATION_ERROR(-20001, '목요일에는 삭제할 수 없습니다.');
```

```
    END IF;
END;
```

```
delete from enrollments where lcode='A109' and scode='92414033';
```

2) 학생 테이블에서 학년을 1~4가 아닌 값을 입력하거나 수정하려면 오류를 발생시킨다.

```
CREATE OR REPLACE TRIGGER TRG05
BEFORE INSERT OR UPDATE OF YEAR ON STUDENTS
FOR EACH ROW
BEGIN
    IF :NEW.year NOT BETWEEN 1 AND 4 THEN
        RAISE_APPLICATION_ERROR(-20001, '학년이 범위를 벗어났습니다. 1~4를 입력하세요.');
```

```
    END IF;
END;
```

```
update students set year=5 where sname='서연우';
```

3) 강좌 테이블(courses)의 강좌번호(lcode)를 소문자로 입력하면 입력 전에 대문자로 자동 변환된다.

```
CREATE OR REPLACE TRIGGER TRG06
BEFORE INSERT OR UPDATE OF LCODE ON COURSES
FOR EACH ROW
BEGIN
    :NEW.lcode := UPPER(:NEW.lcode);
END;
```

```
--강좌번호가 대문자로 변경되어 강좌 테이블에 등록된다.
insert into courses(lcode, lname) values('c302', '자바프로그래밍');
```

4) 수강신청 테이블(enrollments)의 강좌번호(lcode)를 소문자로 입력하면 추가되기 전에 대문자로 자동 변환된다.

```
CREATE OR REPLACE TRIGGER TRG07
BEFORE INSERT OR UPDATE ON ENROLLMENTS
FOR EACH ROW
BEGIN
    :NEW.lcode := UPPER(:NEW.lcode);
END;
```

```
--강좌번호가 대문자로 변경되어 수강신청 테이블에 등록된다.
insert into enrollments(lcode, scode, edate) values('c302', '92414033', sysdate);
```

5) 수강신청 테이블에 데이터를 등록할 경우 수강인원이 최대수강인원을 초과할 경우 에러를 발생한다.

```
--수강신청 테이블에서 강좌별 수강인원 수를 구하여 강좌 테이블 수강인원 수(persons)를 수정한다.
update courses
    set persons=(select count(*) from enrollments where lcode=courses.lcode);

--강좌 테이블 강좌번호 'C302'의 최대수강인원수를 1로 수정한다.
update courses set capacity=1 where lcode='C302';
select * from courses;
```

```
CREATE OR REPLACE TRIGGER TRG08
BEFORE INSERT ON ENROLLMENTS
FOR EACH ROW
DECLARE
    v_capacity courses.capacity%TYPE;
    v_persons courses.persons%TYPE;
BEGIN
    SELECT capacity, persons INTO v_capacity, v_persons FROM courses WHERE lcode=:NEW.lcode;
    IF v_capacity = v_persons THEN
        RAISE_APPLICATION_ERROR(-20001, '수강인원이 초과되었습니다.' || '(최대인원:' || v_capacity || ', 수강인원:' || v_persons ||)');
    END IF;
END;
```

```
--최대수강인원이 1이므로 입력 오류가 발생
insert into enrollments(lcode, scode, edate) values('C302', 92414029, sysdate);

--최대수강인원을 5로 변경한 후 SQL문을 실행하면 입력 성공
update courses set capacity=5 where lcode='C302';
insert into enrollments(lcode, scode, edate) values('C302', 92414029, sysdate);
```

• 자동 데이터 업데이트

1) 수강신청 데이터 입력 후 강좌 테이블의 수강인원 수를 1증가, 수강신청 데이터 삭제 후 수강인원수를 1감소한다.

--수강신청 테이블에서 강좌별 수강인원 수를 구하여 강좌 테이블 수강인원 수(persons)를 수정한다.

```
update courses
  set persons=(select count(*) from enrollments where lcode=courses.lcode);

select * from courses;
```

```
CREATE OR REPLACE TRIGGER TRG09
AFTER DELETE OR INSERT ON enrollments
FOR EACH ROW
BEGIN
  IF INSERTING THEN
    UPDATE courses SET persons=persons+1 WHERE lcode=:NEW.lcode;
  ELSIF DELETING THEN
    UPDATE courses SET persons=persons-1 WHERE lcode=:OLD.lcode;
  END IF;
END;
```

```
insert into enrollments(lcode, scode) values('C302' , '92514009');
delete from enrollments where lcode='C302' and scode='92514009';
```

2) 학생 테이블의 학생을 등록, 삭제, 수정할 때마다 교수 테이블의 지도 학생 수를 수정한다.

--새로운 지도학생 수(students_count) 칼럼을 추가한다.

```
alter table professors add students_count number default 0;
```

--학생 테이블에서 지도교수별 학생 수를 구하여 교수테이블의 지도학생수(students_count)에 저장한다.

```
update professors
  set students_count = (select count(*) from students where advisor=professors.pcode);

select * from professors;
```

```
CREATE OR REPLACE TRIGGER TGR10
AFTER DELETE OR INSERT OR UPDATE ON STUDENTS
FOR EACH ROW
BEGIN
  IF INSERTING THEN
    UPDATE professors SET students_count=students_count+1 WHERE pcode=:NEW.advisor;
  ELSIF DELETING THEN
    UPDATE professors SET students_count=students_count-1 WHERE pcode=:OLD.advisor;
  ELSIF UPDATING THEN
    UPDATE professors SET students_count=students_count-1 WHERE pcode=:OLD.advisor;
    UPDATE professors SET students_count=students_count+1 WHERE pcode=:NEW.advisor;
  END IF;
END;
```

```
insert into students(scode, sname, advisor) values(92414030, '홍길동', 228); --228교수 지도학생 1증가
update students set advisor=221 where sname='홍길동'; --228 지도학생 1감소, 221 지도학생 1증가
delete from students where sname='홍길동'; --221 지도학생 1증가
select * from professors;
```

3) 수강신청, 취소한 경우 학생 테이블의 수강신청 과목 수를 변경한다.

```
-- 학생 테이블에 수강과목 수(courses_count) 칼럼을 추가한다.
alter table students add courses_count number default 0;

-- 수강신청 테이블에서 수강과목 수를 계산하여 학생 테이블에 저장한다.
update students
    set courses_count=(select count(*) from enrollments where scode=students.scode);

select * from students;
```

```
CREATE OR REPLACE TRIGGER TRG11
AFTER DELETE OR INSERT ON ENROLLMENTS
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        UPDATE students SET courses_count=courses_count+1 WHERE scode=:NEW.scode;
    ELSIF DELETING THEN
        UPDATE students SET courses_count=courses_count-1 WHERE scode=:NEW.scode;
    END IF;
END;
```

```
--92424029 학생의 강좌수가 1 감소
delete from enrollments where lcode='C302' and scode=92414029;
--92414029 학생의 강좌수가 1증가
insert into enrollments(lcode, scode) values('C302', 92414029);
select * from students;
```

4) 학생 테이블의 학년을 4학년으로 변경하면 students_history 테이블에 학생 정보를 입력한다.

```
-- 학생 테이블을 구조만 복사하여 stu_history 테이블로 생성한다.
create table students_history as
    select * from students where year=4;

-- stu_history 테이블에 insert_date 칼럼을 추가한다.
alter table students_history add insert_date timestamp default sysdate;

select * from students_history;
```

```
CREATE OR REPLACE TRIGGER TRG12
AFTER UPDATE ON STUDENTS
FOR EACH ROW
BEGIN
    IF :NEW.year = 4 AND :OLD.year !=4 THEN --4학년이 아니면서 4학년으로 변경한 경우만 등록
        INSERT INTO students_history(scode, sname, dept, year, birthday, advisor, insert_date)
            VALUES(:NEW.scode, :NEW.sname, :NEW.dept, :NEW.year, :NEW.birthday, :NEW.advisor, SYSDATE);
    END IF;
END;
```

```
--학생테이블의 학년을 4학년으로 변경하면 students_history에 정보 입력
update students set year=4 where scode='92414029';
select * from students_history;
```